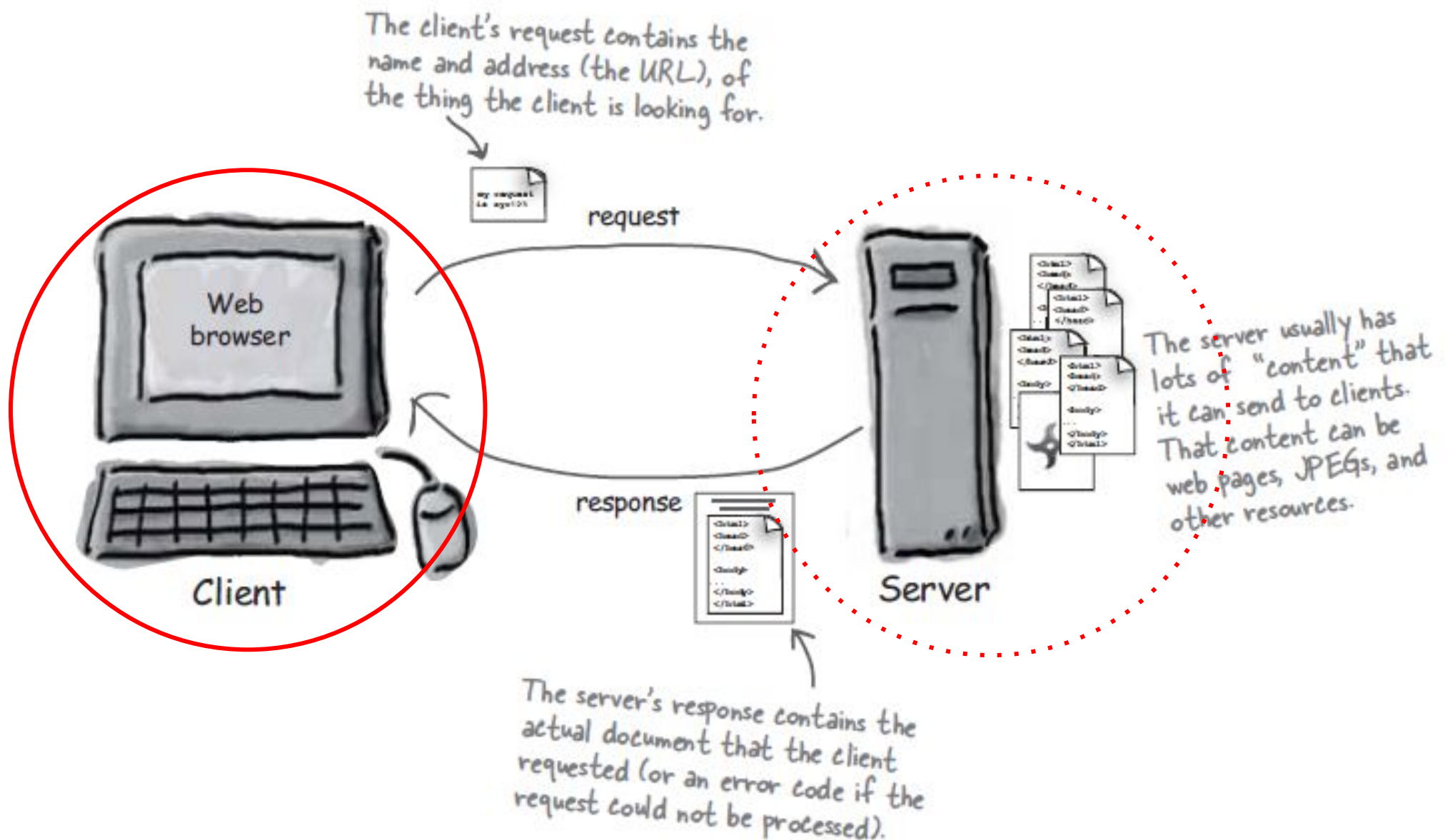


dr. Matej Šprogar

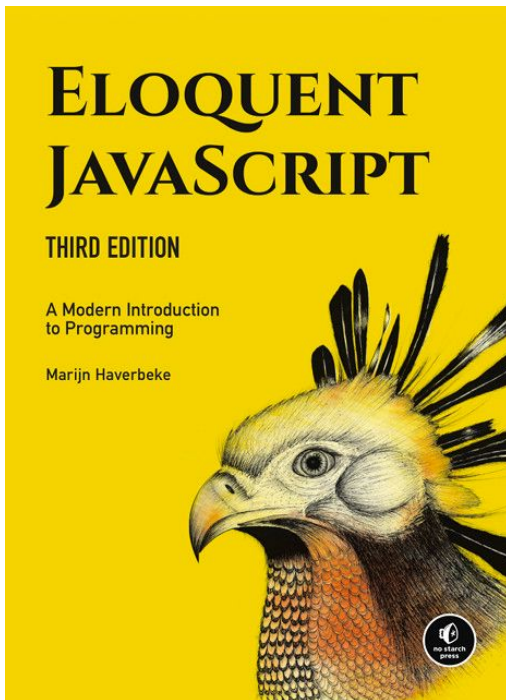
JavaScript

Odjemalec / strežnik arhitektura



Literatura, naloge

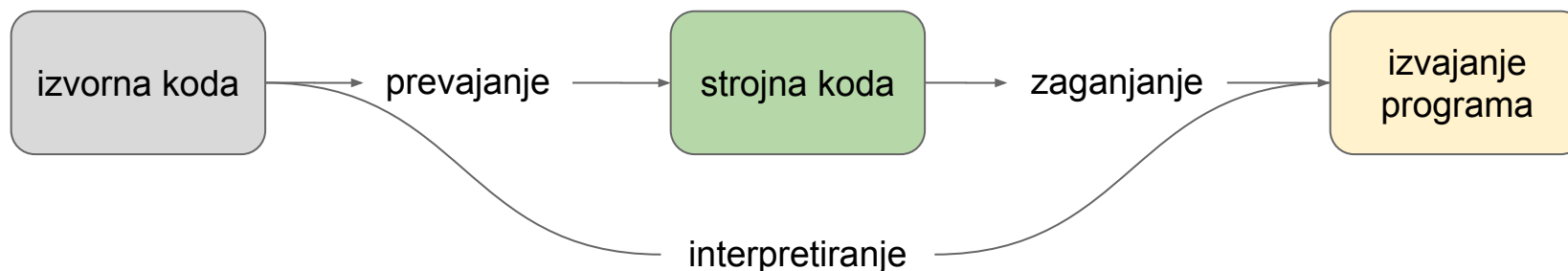
1. <https://eloquentjavascript.net/>
2. www.w3schools.com
3. vaje



Še en programski jezik?

Interpretiran in *omejen* jezik, ki ga v okviru HTML izvaja *brskalnik*. Je že standard na klientu (>97%), na strežnikih je (zaenkrat) majhen (6%).

Interpretiran pomeni, da ga interpreter med "branjem" hkrati tudi že izvaja*:



Splošno podprt, standardiziran, "objektno" naravnan ...

Namen

V okviru HTML omogoča **popoln** nadzor nad HTML dokumentom, ne pa tudi nad računalnikom, sicer bi zlonamerni programerji z njim lahko upravljali z našimi računalniki!

Kot interpretiran jezik ni namenjen pisanju dolge kode, saj ne nudi prevajalniške podpore za preprečevanje napak, ampak se napake zgodijo, ko je že prepozno!

Se razvija in nadgrajuje, osnovna omejitev interpretiranega jezika ostaja, razen če preklopimo na TypeScript, ki pa NI standard, povzroča MS odvisnost...

Zgodovina

Zametki v projektu Netscape, predhodniku Firefox

Politično obarvana preimenovanja:

Mocha -> LiveScript -> JavaScript

Od 1996 standardiziran pri ECMA kot ECMAScript (ES) in JS kot tak je implementacija ES standarda; seveda pa se vse JS implementacije ne držijo standarda...

Najpopularnejši JS interpreterji po brskalnikih:

Quantum (Firefox), V8 (Chrome, Edge, Opera), JSCore (Safari)

JS sintaksa

Jezik je zelo podoben Javi, vendar NI Java, daleč od tega! Enako kot Java se izvaja po vrsticah, ima iste strukturne elemente, omogoča tudi objektno programiranje; tu pa se podobnost že tudi končuje.

Najpomembnejše razlike:

- Spremenljivk ne rabimo najavljati, zgolj uporabimo
- *Weakly-typed* jezik - tip objekta se prilagodi uporabi
- Dvigovanje spremenljivk in funkcij
- First class funkcije - funkcija je vrednost
- Prototipno programiranje brez razredov
- ...

Glavni podatkovni tipi

JavaScript hrani podatke v objektih tipa

- **Number**
x = 42;
- **Boolean**
x = true;
- **String**
x = 'ponedeljek';
y = "torek";
- **Object** (množica parov *ključ: vrednost* v zavutih oklepajih { })
x = {ime: 'Miki miška', letnik: 1928};
 - **Array** (objekt parov index:vrednost v oglatih oklepajih [])
x = [10, 20, 50];
y = Array(10, 20, 50);
z = **new** Array(10, 20, 50);

Uporaba spremenljivk

JavaScript interpreter avtomatsko generira objekte na podlagi podane vsebine:

```
stevilo = 42;                                // number
beseda = "ponedeljek";                        // string
pizza = {                                     // object
    ime: "Klasika",
    obloge: ["šunka", "sir", "gobe"],
    cena: 5.5
};
```

Struktura JavaScript programa

Podobno kot drugi programski jeziki JavaScript omogoča strukturiranje kode s pomočjo programskih blokov in osnovnih ukazov (*if-else, for, while, do-while, break, continue, switch, throw, try-catch, var, let, const, function, return, class*).

Iteracije so dodatno podprte z ukazi *for/of, for/in, for await/of*.

Rahlo tipiziran jezik (*weakly typed*)

Spremenljivka lahko hrani karkoli (*number, String, Object, ...*) in tip vsebine se lahko spreminja! JavaScript sam ugotavlja, kakšnega tipa bi **naj** bili podatki v spremenljivki:

```
alfa = "nekaj";  
alfa = 123;
```

To omogoča recimo:

```
cena = 12.75;  
cena = cena + "€";
```

Vsiljevanje tipov

Rahlo tipizirani jeziki omogočajo prisilno spreminjanje tipa (type coercion):

$1 + \text{null} == 1$

$"1" - 1 == 0$

$"1" + 1 == "11"$

$"beseda" * 1 == \text{NaN}$

$\text{null} == \text{undefined}$

$\text{null} !== \text{undefined}$

$\text{null} != 0$

Najava: *var*, *let*, *const*

- **var**

Globalni/funkcijski doseg, večkratna deklaracija dovoljena, globalne spremenljivke postanejo lastnost *window* object-a, zato jih lahko tudi izbrišemo (*delete*);

- **let**

Nova spremenljivka z *blokovnim* dosegom, večkratna in kasnejša deklaracija prepovedana, ne sme redefinirati var-a, *delete* ni mogoč;

- **const**

Blokovni doseg brez možnosti sprememb, večkratna deklaracija prepovedana, *delete* ni mogoč

Najava/uporaba spremenljivk

Spremenljivko lahko sicer uporabimo brez najave ali še PREDEN smo jo najavili z *var*. JS *dvigne (hoisting)* deklaracijo spremenljivke (ali funkcije, razreda ali importa) na vrh programa/bloka še pred izvršitvijo...

```
↑ alfa = stevilo; // undefined  
  var stevilo;  
  
  beta = gama;    // ReferenceError  
  let gama;
```

Najavi z `let` in `const` tega ne omogočata, saj vplivata na celi blok.

Najava spremenljivk je smiselna, ker preprečuje problem prekrivanja imen:

```
x = 42;  
funkcija_ki_spremeni_x();    // x = 3  
console.log(x);             // 3!?
```

Različne najave:

Ista spremenljivka:

```
var i = 5;  
for (var i=0; i<7; i++) {  
    // ukazi  
}  
// i == window.i == 7  
let i = 3;    // error
```

Dve spremenljivki:

```
let i = 5;  
for (let i=0; i<7; i++) {  
    // ukazi  
}  
// i == 5  
let i = 3;    // error
```

Const spremenljivke ne moremo ne spremeniti ne ponovno najaviti!

```
const i = 5;  
i = 51;        // error  
const i = 51;  // error
```

Funkcije

Funkcije so poimenovani bloki kode. Lahko jih definiramo na več načinov:

```
function alfa() {...}    // definicija
```

```
alfa = function() {...}; // izraz (expression)
```

Ime *alfa* sedaj predstavlja blok funkcijske kode.

```
alfa;           // function alfa()
```

Funkcijo kličemo z njenim imenom in okroglimi oklepaji, med katere lahko dodamo morebitne parametre:

```
alfa();         // klic funkcije alfa!
```

```
beta(7, "ena"); // klic funkcije beta
```


Funkcija, *first-class citizen*

Funkcijo lahko obravnavamo kot podatek - jo shranimo v spremenljivko, posredujemo kot parameter...

```
function vrni10() { return 3+7; }
```

```
function logiraj(kaj) { console.log(kaj()); }
```

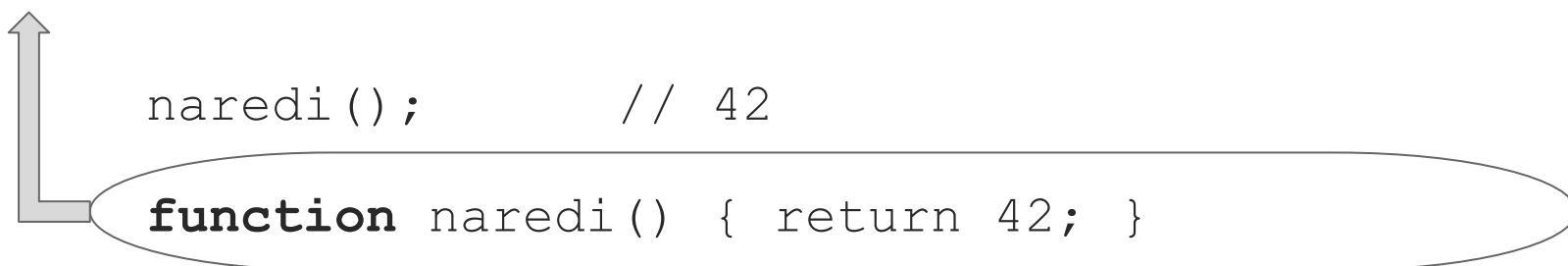
```
y = vrni10(); // y = 10
```

```
f = vrni10; // f = funkcija
```

```
logiraj( vrni10 ); // posredujemo
```

Dviganje: funkcije

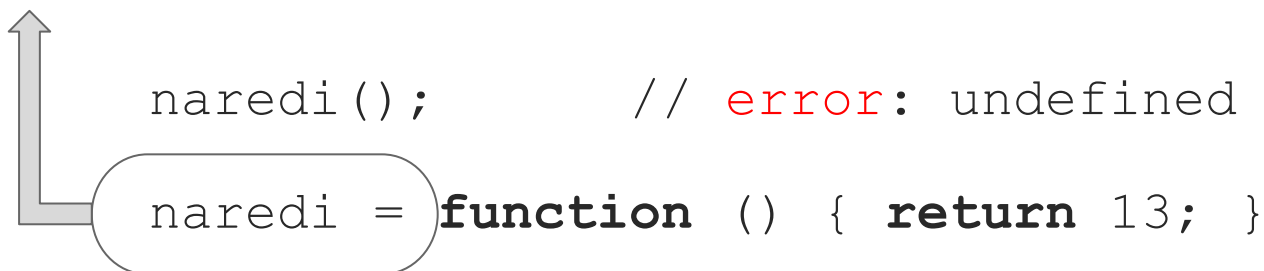
Funkcijo lahko kličemo, še preden smo jo definirali:



A diagram illustrating a function call before its definition. A large grey L-shaped arrow points from the function call line to the function definition line. The function call is `naredi(); // 42`. The function definition is `function naredi() { return 42; }`, which is enclosed in a rounded rectangle.

```
naredi(); // 42  
function naredi() { return 42; }
```

Dviganje funkcije velja samo za *definirane* funkcije; če funkcijo naredimo z izrazom (*expression*) in shranimo v spremenljivko, se bo dvignila le spremenljivka, ki pa bo pri klicu *undefined*!



A diagram illustrating a function call before its definition with an assignment. A large grey L-shaped arrow points from the function call line to the function definition line. The function call is `naredi(); // error: undefined`. The function definition is `naredi = function () { return 13; }`, where the assignment part is enclosed in a rounded rectangle.

```
naredi(); // error: undefined  
  
naredi = function () { return 13; }
```

Arrow funkcije

Tretji način deklaracije funkcij je s *puščico* namesto s *function*:

```
alfa = () => { ... };
```

```
alfa = (param1, param2) => { ... };
```

```
alfa = (param1) => { return param1+obj2; };
```

```
alfa = param1 => param1+obj2;
```

V telesu lahko uporabimo vse v trenutku klica veljavne in dosegljive zunanje objekte (*capture*).

Arrow funkcije so enakovredne klasičnim funkcijam, nimajo pa **svoje** *this* reference. Dejansko so zelo podobne tim. *lambda* funkcijam v drugih jezikih.

Lastnosti (*Properties*)

JavaScript objekti (z izjemo *null* in *undefined*) sestojijo iz izbranih lastnosti, ki hranijo vsebino, opis ali funkcijo objekta.

Imena lastnosti so **stringi**, zato lahko do lastnosti dostopamo ali s pomočjo operatorja pika ali z oglatimi oklepaji; slednji način je univerzalen, pika je zgolj priročnejša:

```
obj.lastnost === obj['lastnost']
```

```
mm = {ime:'Miki', priimek:'Miška'};
```

```
console.log(mm.ime);           // kratko
```

```
console.log(mm['ime']);        // univerzalno
```

Lastnosti ...

Objekti imajo lastnosti z vrednostmi, ki jih lahko spreminjamo, dodajamo in brišemo:

```
mm = {ime:'Miki', priimek:'Miška',  
      letnik:1928};  
  
mm.ime = 'Mickey';           // spremeni  
  
mm['priimek'] = 'Mouse';  
  
  
mm['partner'] = 'Mini';      // doda  
  
mm.avtor = 'Walt Disney';  
  
delete mm.letnik;           // briše lastnost  
  
mm.year = 1928;              // doda
```

Objektne lastnosti ...

Če je ime lastnosti *pravilno* računalniško ime, lahko do lastnosti dostopamo s piko, sicer z oglatimi oklepaji!

```
obj = {1:'abc', "2":'def', 'dolgo ime':7};
```

```
obj.1 // napaka
```

```
obj['1'] === obj[1] // enakovredno!
```

```
obj['2'] === obj[2] // enakovredno!
```

Priročna je kratka notacija:

```
params = [1,2,3];
```

```
naloga = {params}; // {params:params}
```

```
naloga = {[params[0]]:10}; // {1:10} ES6
```

Metode (*Methods*)

Lastnosti lahko hranijo tudi funkcije; v tem primeru jim pravimo "*metode*", shranimo in kličemo jih običajno:

```
obj = {};
```

```
obj.funkcija = function() { ... };
```

```
obj.funkcija();           // klic metode
```

Metode

Objektu lahko dodamo metodo na več načinov:

// 1:

```
function odgovor() { return 42; }  
problem = {leto:1979, odgovor};
```

// 2:

```
problem = {leto:1979, odgovor(){ return 42;}};
```

// 3:

```
problem = {leto:1979, odgovor: function() {  
    return 42; }};
```

// 4:

```
problem = {leto:1979};  
problem.odgovor = function() { return 42; };
```


Metode + *this*

Vsaka metoda (razen(!) *lambde*) ima lastnost *this*, ki referencira klicajoči objekt (default je *window* object).

```
function aloalo() {  
    console.log('It is I, ' + this.ime);  
}  
  
let obj = {ime:"Leclerc", aloalo};  
obj.aloalo();
```

Lahko pa uporabimo *call* metodo; prvi parameter bo *this*:

```
aloalo.call(obj);
```

Prototipi vs razredi

Sprva JS ni direktno podpiral razredov, ampak tim. *prototipe*. Razredi so pravzaprav posebne funkcije, a jih moramo za razliko od spremenljivk in funkcij definirati *pred* uporabo:

```
class Kamion {} // razred
```

```
const tamic = new Kamion(); // objekt
```

Razrede lahko ustvarimo tudi s tim. razrednimi izrazi (*class expressions*):

```
let Kamion = class {} // razred
```

```
let Avto = class Avtomobil {} // razred
```

```
let tamic = new class {} // objekt
```

Razredi (Classes) [Mozilla](#)

JavaScript razred je zgolj priročna notacija za ustvarjanje prototipov, ki izhaja iz originalne JS zasnove.

Definicija razreda *lahko* vsebuje le funkcije (constructor + metode), saj lahko attribute inicializirane *dodamo* v konstruktorju skozi *this*.

```
class Problem {  
    constructor() { this.leto = 1979; }  
    odgovor() { return 42; }  
}
```

```
problem = new Problem();  
problem.odgovor();           // 42  
console.log(problem.leto);   // 1979
```

Konstruktorska funkcija

Lahko pa se izognemo celotni kolobociji z razredom in eksplicitno *constructor* metodo, saj lahko objekt ustvarimo kar direktno z operatorjem *new* in *konstruktorsko* funkcijo:

```
function Problem(leto) {  
    this.leto = leto;  
}  
  
let alkaida = new Problem(2001);  
let korona = new Problem(2019);
```

Nastale objekte lahko sedaj normalno uporabljamo, jim dodajamo ali odvzemamo (*delete*) attribute...

Get in set metodi

```
class Problem {  
    constructor(year) {  
        this.year = year;  
    }  
    get leto() {  
        return "Piše se leto " + this.year;  
    }  
    set leto(value) {  
        this.year = value;  
    }  
}  
  
let korona = new Problem(2019);  
console.log(korona.leto);    // Piše se leto 2019  
korona.leto = 2020;
```

Koristne metode

Array.prototype:

push, pop, forEach, map, every, reduce, slice ...

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array

Object.prototype:

create, is, assign, toString ...

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object

Posebnosti

```
4 == 4, 4 == '4', 4 !== '4'
```

```
NaN !== NaN, Object.is(NaN, NaN)
```

```
for (let elt of "abc") {} // a, b, c
```

```
for (let elt in "abc") {} // 0, 1, 2
```

for/of != for/in

for/in ... sprehod skozi *lastnosti* **objekta**

for/of ... *vrednosti* v iterabilnem **zaporedju**

```
for (let [key, value] of Object.entries(obj)) {}
```

immutable numbers, Booleans, strings:

```
let abc = "abc"; abc.x = 2; abc.x == undefined
```

OOP: enkapsulacija, polimorfizem, dedovanje...

JS na svoj način omogoča tudi objektno programiranje. Enkapsulacija je mogoča z *let* closure ali *#* notacijo, dedovanje razredov je podprto z magičnima besedama *extends* in *super*.

Ker objekti "dedujejo" metode prototipov je polimorfizem, podobno kot pri Javi, vedno prisoten - jezik vedno kliče "pravo" objektno metodo, saj druge opcije nima. Pogoji seveda je, da objekt implementira pričakovani API.

JS vs. Java/C#/C++

Če lahko JS objektom attribute kadarkoli dodamo ali odstranimo, so klasični objekti nespremenljivi, spremembe "živega" tipa pa nemogoče (dodajanje atributov kvečjemu skozi dedovanje, brisanje ni mogoče).

Klasično objekt vedno naredimo s pomočjo razrednega tipa, v JS pa praviloma neposredno s konstruktorsko funkcijo; razredi v JS omogočajo le drugačen opis sicer kompliciranih objektov.

Brez tipov preverjanje podatkov v spremenljivkah ni mogoče. Prevajalnik lahko prepreči napačno uporabo napačnih objektov in metod v napačnem trenutku; JS nima teh varovalk in lahko nasprotno pomeša hruške in jabolke...

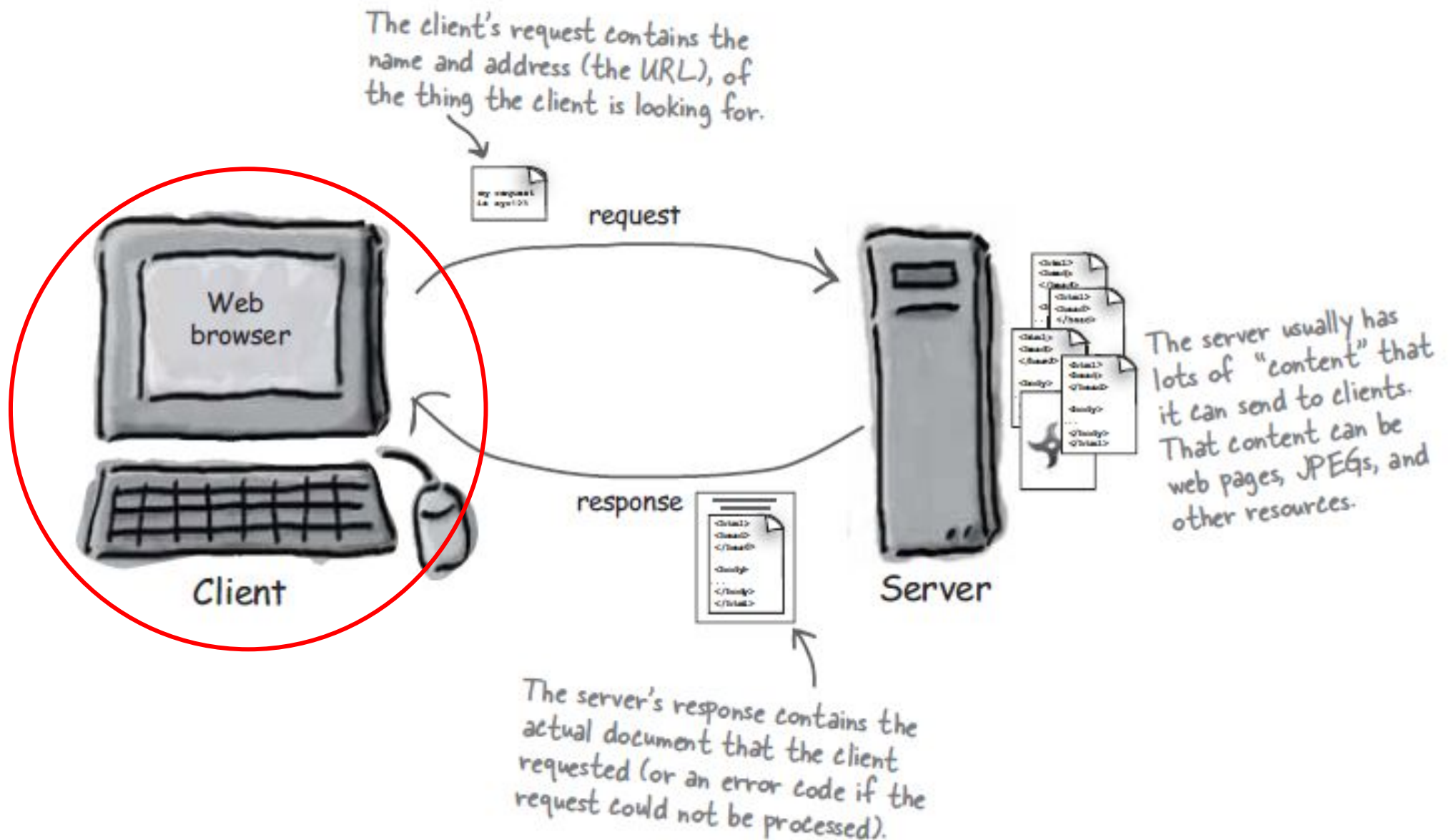
Programiranje s tipi zahteva več znanja, da lahko prevajalniku dopovemo, kaj točno želimo in česa ne želimo; enostavnejši JS tega ne omogoča.

JavaScript @ HTML



integracija

Javascript v HTML



JavaScript

Ukazi so lahko vključeni ali neposredno v dokumentu (v glavi ali telesu) s pomočjo `<script>...</script>` elementa ali preko povezane zunanje datoteke (`<script src="myscript.js"></script>`).

```
<!doctype html>
<html lang="sl">
  <head><meta charset=utf-8></head>
  <body>
    <script type="text/javascript">
      document.write("Hello, World!")
    </script>
  </body>
</html>
```

Lahko pa tudi takole...

// halo.html

```
<!doctype html>
<html lang="sl">
  <head><meta charset=utf-8></head>
  <body>
    <script src="halo.js"></script>
  </body>
</html>
```

// halo.js

```
document.write("Hello, World!")
```

Nalaganje JavaScripta

Brskalnikov postopek nalaganja strani s `<script>` oznako:

1. pridobi HTML kodo
2. prične z razpoznavo HTML
3. če med branjem HTML naleti na `<script src="...">` oznako:
 - a. zahteva omenjeno zunanjo datoteko
 - b. ***preneha(!)*** z razpoznavo HTML dokler ni koda dostavljena
 - c. prebere in izvede JS ukaze
4. nadaljuje z razpoznavo preostalega HTML dokumenta

Pri nalaganju zunanjega skripta lahko zato izberemo “async” ali “defer” opcijo, ki obe brskalniku dovolita neprekinjeno razpoznavo HTML! Npr.:

```
<script src="demo_async.js" defer></script>
```

async: JS izveden *takoj* po prejemu skripta, vrstni red vseh skript vprašljiv

defer: izvedeno v izbranem vrstnem redu *na koncu* html dokumenta

Tretji pristop...

```
<!doctype html>

<html>

  <meta charset=utf-8>
  <title>Drugi način</title>

<body>
  <p id="message">No JavaScript Available</p>
  <script>
    document.getElementById("message")
      .innerHTML = "Hello World!";
  </script>

</body>
</html>
```

Vrstni red (<p> in <script>) je pomemben!

Možnosti...

```
<p id="message">Ni JavaScripta!</p>
<script>
    let elt = document.getElementById("message");
    elt.pripraviVsebinsko = function () {
        this.innerHTML = "Dober dan!";
    }
    elt.pripraviVsebinsko();
    pripraviPisavo(elt);
    function pripraviPisavo(par) {
        par.style.fontSize = "40px";
    }
    pripraviOzadje.call(elt);
    function pripraviOzadje() {
        this.style.backgroundColor = "rgba(0,0,255,0.7)";
    }
</script>
```


Dogodkovno voden jezik

JavaScript je dogodkovno-voden jezik. Funkcijo tipično sproži predviden dogodek - nobena funkcija ni v pogonu ves čas.

```
<p id="message" onclick="goodbye('world')">Ni JavaScripta!</p>
```

```
<script>
```

```
  let elt = document.getElementById("message");  
  elt.innerHTML = "Halo in klikni name!";  
  // elt.onmouseover = function() { goodbye("OST"); };  
  elt.addEventListener("mouseover",  
    function(){ goodbye('OST') }  
  );  
  function goodbye(who) {  
    elt.innerHTML = "Goodbye " + who;  
    elt.style.backgroundColor = "lightblue";  
  }
```

```
</script>
```

Javascript dogodki

onabort, onbeforeunload, onblur, onchange,
onclick, ondblclick, onerror, onfocus, onkeydown,
onkeypress, onkeyup, onload, onmousedown,
onmousemove, onmouseout, onmouseover, onmouseup,
onreset, onresize, onselect, onsubmit, onunload

<http://www.w3schools.com/js/>

https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Building_blocks/Events

Kaj se bo zgodilo? Poenostavimo...

```
<!DOCTYPE html>
<head lang=sl>
  <meta charset="utf-8">
  <title>JS example</title>
  <script>
    function setup() {
      let inputs =
        document.getElementsByTagName('input');
      for (let i = 0; i < inputs.length; i++) {
        inputs[i].addEventListener('mouseover',
          function (evt) { evt.target.focus(); });
      }
    }
    window.onload = setup;
  </script>
</head>
<body>
  Name: <input><br>Email: <input>
</body>
</html>
```

DOM

DOM - Document Object Model

HTML je predstavljen kot drevesna struktura objektnih vozlišč, ki jo je enostavno brati in manipulirati.

```
elt = document.getElementById("prvi");  
elt.classList.add("aktiven"); // !IE  
elt.className = elt.className + " aktiven";
```

Web API

Za lažje pisanje kode na spletu je na voljo veliko tim. programerskih vmesnikov, ki omogočajo *enostavnejše* delo s pomočjo problemsko prirejenih objektnih tipov s preprostimi vmesniki za delo z njimi.

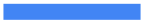
Glej <https://developer.mozilla.org/en-US/docs/Web/API>

Omejitve JavaScripta v brskalniku

1. Ne more pisati po datotekah na strežniku brez pomoči posebne strežniške skripte z ustreznimi dovoljenji
2. Nima dostopa do podatkovne baze na strežniku
3. Ne more brati ali pisati datotek na klientu
4. Ne more zapreti okna, ki ga ni odprl
5. Ne more dostopati do strani v drugi domeni
6. Ne more zaščititi vsebine strani ali slik na strani pred prenosom

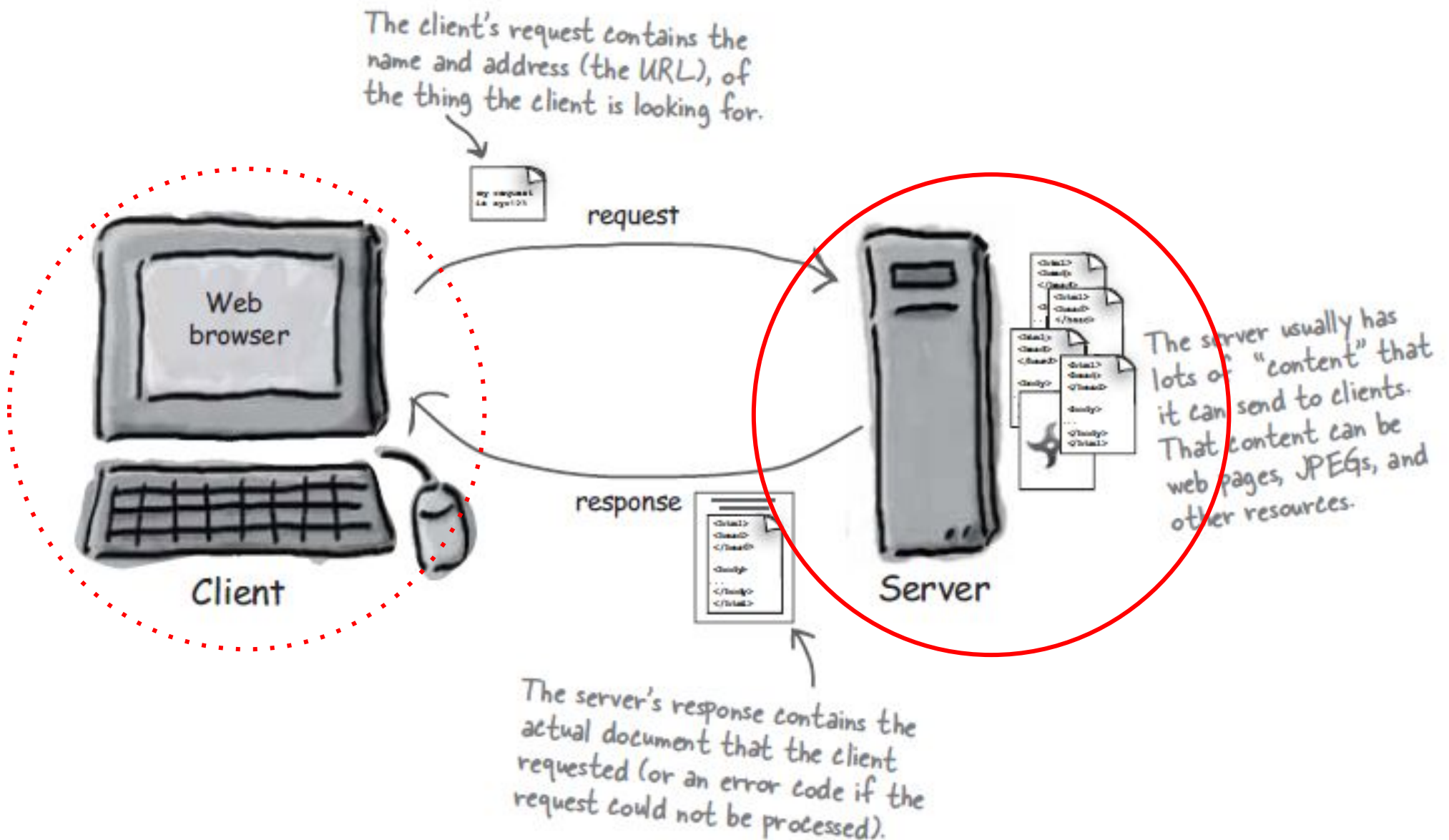
Source: <https://www.thoughtco.com/what-javascript-cannot-do-2037666>

Generiranje HTML



Kako sploh ustvarimo množico HTML-jev?

Kje nastane/hranimo HTML



Priprava HTML

Večinoma HTML strani izdelamo ali (1) ročno ali (2) programsko vnaprej ali jih ustvarimo (3) programsko sproti. Strani, ki so pripravljene vnaprej in so za vse uporabnike enake, imenujemo tudi **statične** strani; nasprotno so bralcu prilagojene strani generirane na zahtevo - **dinamične** strani.

Statična stran je lahko čisti HTML ali HTML+JS; slednja opcija omogoča dinamično *obnašanje* tudi statičnim stranem! Statične strani lahko zgradimo tudi s pomočjo generatorja statičnih strani (SSG - Static Site Generator).

Dinamično stran, ki je tudi lahko čisti HTML ali HTML+JS, lahko programsko generiramo na strežniku (*Server-Side-Rendering SSR*) ali klientu (*Client-Side-Rendering CSR*). To je tipično naloga tim. *template* pogona (*template engine*), ki na podlagi statične predloge in *dinamično* pridobljenih podatkov generira ustrezni HTML+JS.

Dinamični HTML

Dinamično spletno stran z uporabniku prilagojeno vsebino lahko torej dosežemo na več načinov, najbolj pogosta sta:

1. server-side (SS) rendering z ustvarjanjem več strani
2. client-side (CS) rendering s spreminjanjem *ene* strani

Seveda lahko tudi kombiniramo: CS rendering + remote-scripting dostop do strežnika ali SS rendering + JS na klientu.

Pogosta oblika spletnih aplikacij je *Single Page Application* (SPA), kjer je v HTML sprva zgolj prazen prostor za vsebino, ki se nato dinamično napolni s pomočjo JavaScript kode na strani klienta (CS). Takšna zasnova ne potrebuje osveževanja strani, vsa potrebna koda pa je tipično naložena že na začetku ali po potrebi.